

Maximisation Bias in Tabular Q-learning and Double Q-learning: Bias and Policy Cost on a Trap MDP and Random MDPs

Pollux-75L

Stemma seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/overestimation-q-learning-vs-double-q-learning>

Abstract—Q-learning bootstraps from the maximum of noisy value estimates, which is positively biased; double Q-learning removes this bias by decoupling action selection from evaluation. We reimplement tabular Q-learning, Double Q-learning, Weighted Double Q-learning and Maxmin Q-learning in numpy, vectorised over 1,000 independent runs per seed (5 seeds), and measure the estimated-minus-true value of the start state and the fraction of suboptimal actions on the Sutton–Barto maximisation-bias MDP and on random 20-state MDPs. On the trap MDP the textbook picture holds: the peak start-state overestimate is 0.1060 for Q-learning and 0.008322 for Double Q-learning, and the suboptimal-action fraction falls from 0.3795 to 0.1186. On random MDPs the picture is mixed: every method underestimates the start-state value at the end of training, Q-learning least (-0.1498 versus -0.3636 for Double Q-learning), and Double Q-learning takes more suboptimal actions (0.3119 versus 0.2551). A warm-start ablation, a step-size sweep and an exploration sweep are consistent with this being a finite-time effect of slower per-table learning and of adaptive data collection: with a warm start Double Q-learning takes fewer suboptimal actions, and with uniformly random behaviour the sign of Q-learning’s bias is positive. These are small tabular experiments with one MDP family and fixed hyper-parameters; they do not transfer directly to deep RL.

Index Terms—Q-learning, double Q-learning, maximisation bias, estimation bias, tabular reinforcement learning, reproduction study

I. Introduction

Q-learning estimates action values by bootstrapping from $\max_{a'} Q(s', a')$. When the estimates are noisy, the maximum of estimates is on average larger than the maximum of the true values, an instance of the optimizer’s curse [1]. The resulting overestimation can slow learning, mislead action selection, and in function-approximation settings compound across bootstrapping steps. Double Q-learning (van Hasselt, 2010) keeps two estimators and uses one to pick the maximising action and the other to evaluate it, which removes the positive bias but can introduce a negative one [2], [3]. The idea carries into

deep RL as Double DQN [4], is part of Rainbow [5], and, as clipped double Q-learning, of TD3 [6].

The standard evidence for the benefit of decoupling in the tabular setting is a single small MDP from Sutton and Barto’s textbook (Example 6.7), in which the first action taken from the start state leads either to a terminal state with zero reward or to a state with many actions whose rewards are zero-mean-ish noise with a slightly negative mean. Beyond this example, the questions a practitioner would ask are less well answered in one place: how large is the bias on MDPs without a hand-built trap, does reducing it translate into better action choices at a fixed training budget, and how much of the observed difference between Q-learning and double estimators is due to decoupling rather than to confounds such as each table of a double learner being updated half as often, or the estimates not having converged from their initial values?

We run a controlled tabular study with exact true values. Our contributions are modest and empirical: (i) a vectorised numpy implementation of four estimators with identical step size, exploration and budget, evaluated with 1,000 independent runs per seed; (ii) a measurement of the estimated-minus-true start-state value and of the suboptimal-action fraction on the trap MDP and on random 20-state MDPs; (iii) a set of ablations (warm start at the true values, step size, exploration rate, noise level, number of actions, and updating both tables on each sample) that separate the effect of decoupling from learning speed and from adaptive sampling. The headline result is mixed: the trap-MDP result reproduces, but on random MDPs the double estimator underestimates more than Q-learning and takes more suboptimal actions at our training budget. We state our hypotheses and tests in Section IV and give a verdict for each in Section V. This is a small CPU study; Section VII lists what it does not cover.

II. Related Work

a) Maximisation bias and double estimators.: The positive bias of the maximum of noisy estimates is classical in decision analysis [1]. van Hasselt (2010) introduced

Double Q-learning and the trap MDP of Sutton and Barto’s textbook that we use (neither has an arXiv or Crossref record, so they are not in our reference list). Double DQN [4] transfers the idea to deep networks using the target network as the second estimator. Weighted Double Q-learning [2], whose abstract notes that double Q-learning sometimes underestimates action values, interpolates between the single and the double estimator with a data-dependent weight; we reimplement it as a baseline. Maxmin Q-learning [7] bootstraps from the minimum over several estimators, with the number of estimators controlling the bias, and its abstract highlights that the effect of overestimation on learning efficiency is environment dependent, a point our random-MDP results echo.

b) Underestimation and bias control.: Ren et al. [3] argue that the underestimation of double Q-learning can lead to non-optimal fixed points under an approximate Bellman operator. Cini et al. [8] study a weighted estimator in deep RL, Sabry and Khalifa [9] use dropout to reduce variance and overestimation, and Kuznetsov et al. [10] propose automatic selection of bias-control hyper-parameters for algorithms that include Maxmin Q-learning. Wagenbach and Sabatelli [11] study how learning rate, discount and reward signal influence Q-learning’s overestimation in settings that require function approximation, which motivates our step-size and noise sweeps in a setting where exact values are available. TD3 [6] and Rainbow [5] are examples of systems in which such corrections are standard components.

c) Gap.: Most of the above evaluates new estimators, often with function approximation. We instead ask a narrow, controlled question with exact ground truth: in tabular settings with matched hyper-parameters, how do the standard estimators compare, and which confounds explain the differences? We do not propose a new estimator.

III. Method

All agents are tabular, off-policy and ϵ -greedy. Let $Q_k(s, a)$, $k = 1..K$, be the tables of an agent ($K = 1$ for Q-learning, $K = 2$ otherwise), α the step size, γ the discount, and $\mathcal{A}(s)$ the valid actions. The behaviour values are $\bar{Q} = Q_1$ (Q-learning), $\bar{Q} = \frac{1}{2}(Q_1 + Q_2)$ (the double family) or $\bar{Q} = \min_k Q_k$ (Maxmin); the agent takes a uniformly random valid action with probability ϵ and $\arg \max_a \bar{Q}(s, a)$ otherwise. After a transition (s, a, r, s') the update is $Q(s, a) \leftarrow Q(s, a) + \alpha(y - Q(s, a))$ with target y and $y = r$ if s' is terminal.

a) Q-learning (reimplemented).:

$$y = r + \gamma \max_{a'} Q_1(s', a'). \quad (1)$$

b) Double Q-learning (reimplemented).: A table $u \in \{1, 2\}$ is drawn uniformly per step, o is the other table, and

$$a^* = \arg \max_{a'} Q_u(s', a'), \quad y = r + \gamma Q_o(s', a^*), \quad (2)$$

and only $Q_u(s, a)$ is updated. Each table is therefore updated on about half of the samples.

c) Weighted Double Q-learning (reimplemented).: With $a^* = \arg \max_{a'} Q_u(s', a')$, $a_L = \arg \min_{a'} Q_u(s', a')$ and

$$\beta = \frac{|Q_o(s', a^*) - Q_o(s', a_L)|}{c + |Q_o(s', a^*) - Q_o(s', a_L)|}, \quad y = r + \gamma[\beta Q_u(s', a^*) + (1 - \beta)Q_o(s', a^*)] \quad (3)$$

with $c = 1$ [2]; $\beta \rightarrow 1$ recovers Q-learning’s single estimator and $\beta \rightarrow 0$ Double Q-learning.

d) Maxmin Q-learning (reimplemented, $N = 2$).:

$$y = r + \gamma \max_{a'} \min_k Q_k(s', a'), \quad (4)$$

and one table chosen uniformly is updated [7].

e) Simultaneous double update (ablation).: Both tables are updated on every sample with the cross targets of Eq. (2) (table 1 first, table 2 second, in place). If both tables start equal and receive the same sample, the cross target equals the single-estimator target, so by induction they stay equal and the agent is Q-learning; the in-place order is the only source of difference from Q-learning when a state transitions to itself.

f) Quantities measured.: The estimated value of a state is $\hat{V}(s) = \max_{a \in \mathcal{A}(s)} \bar{Q}(s, a)$; the true optimal value $V^*(s)$ is known (closed form on the trap MDP, value iteration on random MDPs). The start-state bias is $\hat{V}(s_0) - V^*(s_0)$, measured at the end of every episode and averaged over the 1,000 runs. The Q-bias is $Q(A, \text{left}) - q^*(A, \text{left})$ on the trap MDP and $\bar{Q}(s, a) - q^*(s, a)$ averaged over all state-action pairs on random MDPs. The suboptimal-action fraction of an episode is the fraction of actions taken (including exploratory ones, so the floor is not zero) that differ from the optimal action: at the start state on the trap MDP, and over all steps on random MDPs. Greedy error is the fraction of runs (trap) or of states (random) at which the greedy action of $\bar{Q}$ is not optimal. Because $q^*(A, \text{right}) = 0$ is learned exactly, $\hat{V}(A) \geq 0 = V^*(A)$ on the trap MDP: this metric can only register overestimation there, and Q-bias is the signed companion.

IV. Experimental Setup

a) Tasks.: Trap MDP (maxbias). State A is the start; action right ends the episode with reward 0, action left moves to state B with reward 0. State B has 10 actions, each ending the episode with reward drawn from $\mathcal{N}(-0.1, 1)$; $\gamma = 1$. Hence $V^*(A) = 0$ (go right) while $q^*(A, \text{left}) = -0.1$. Episodes: 300. Random MDP (random20). 20 states and 4 actions; each state-action pair has 3 distinct random successor states with Dirichlet(1) probabilities and a fixed mean reward drawn from $\mathcal{N}(0, 1)$; the observed reward adds $\mathcal{N}(0, \sigma^2)$ noise with $\sigma = 1$; $\gamma = 0.7$. An episode has 25 steps from a uniformly random start state; 1,000 episodes per run. State 0 is the designated start state for the bias metric. Q^* is computed by value iteration for every run’s MDP.

b) Runs and seeds.: Each rh run is one seed and contains 1,000 independent replicas (own tables, own noise, and, for random20, its own random MDP), vectorised in numpy. We use 5 seeds (0–4); statistics treat the seed mean over the 1,000 replicas as one observation, so all tests have $n = 5$. Metrics: the final value is the mean over the last 20 episodes, peak is the maximum over episodes of the replica-mean start-state bias, and subopt_all is the suboptimal-action fraction averaged over all episodes (Section III).

c) Hyper-parameters and tuning.: All systems share $\alpha = 0.1$, $\epsilon = 0.1$, zero initialisation, the same episode budget and the same seeds (the same seed index generates the same random MDPs for every system), taken from the Sutton–Barto setting rather than tuned; no system was tuned. α , ϵ , σ and the number of actions are varied only as sensitivity sweeps and never used to select the headline configuration. Weighted Double uses $c = 1$ and Maxmin uses $N = 2$ estimators.

d) Configuration choice and pilot.: The random-MDP configuration ($\gamma = 0.7$, 25-step episodes from random start states, 1,000 episodes) was fixed after a seed-0 pilot, run before any logged result, with $\gamma = 0.9$ and 300 episodes, in which all estimators were still dominated by the transient from zero initialisation. The pilot also suggested that the start-state bias on random MDPs would be negative, so H3 was written knowing this. The pilot was not used to choose among methods.

e) Hypotheses and registered tests.: H1: on the trap MDP Q -learning overestimates $q^*(A, \text{left})$ and Double Q -learning has a lower final Q -bias (Welch test on `qbias_final`, $n = 5$). H2: on the trap MDP Double takes fewer suboptimal actions (`subopt_all`). H3: on random20 Q -learning’s final start-state bias is positive and Double removes it, and Double does not take fewer suboptimal actions (`bias_final`, `subopt_all`). H4: Q -learning’s bias grows with σ and with the number of actions, and Double stays below it (sweeps; read only at grid values). Significance level 0.05; tests are the Welch p -values of rh compare, with Double Q -learning as the reference (differences are Double minus the other system).

f) Ablations and sweeps.: Warm start at the true Q^* (isolates steady-state from transient bias); simultaneous double update; $\sigma \in \{0.25, 1, 2, 4\}$; $\epsilon \in \{0.1, 0.3, 1.0\}$ ($\epsilon = 1$ is uniformly random, non-adaptive behaviour); $\alpha \in \{0.02, 0.05, 0.1, 0.2\}$ on random20, and $n \in \{2, 5, 10, 25, 100\}$ actions at B on the trap MDP. The sweeps run Q -learning and Double Q -learning on random20, and all four systems on the trap MDP, 5 seeds each; the default point of every sweep is taken from the main runs (not repeated).

g) Compute.: The longest single run took 15 seconds on a shared CPU (2 threads), and the 250 experiment runs together sum to 24 minutes of run time. Code: `method/run.py` (committed before the main runs), `experiments/jobs.py` generates the run list, analy-

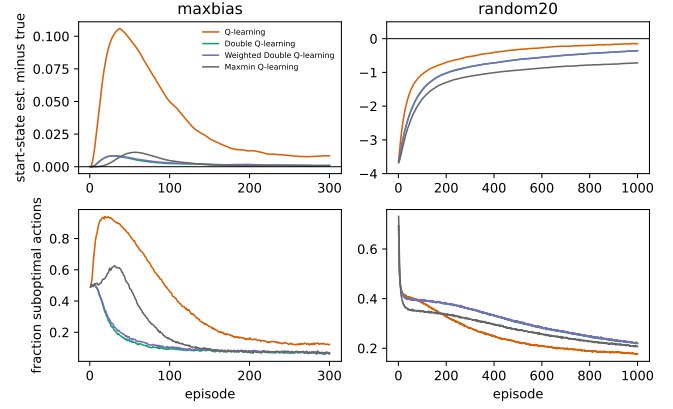


Fig. 1. Top: start-state estimated-minus-true value; bottom: suboptimal-action fraction per episode. Left: trap MDP; right: random20 (y-axis clipped at -4 to 0.5). Orange: Q -learning, green: Double, purple: Weighted Double, grey: Maxmin; Double and Weighted Double nearly coincide.

sis/make_sweeps.py aggregates the sweeps from the run registry.

V. Results

Table I gives the main comparison and Fig. 1 the learning curves (mean over seeds of the 1,000-run means). Bold and underline in the generated tables follow each metric’s declared direction (lower is better) and are therefore not meaningful for the signed bias columns; we read the signed numbers directly. Unless stated, p is the Welch p -value over 5 seeds with Double Q -learning as reference.

a) H1 (trap MDP, Q -bias): supported in direction, not in the strong form.: The registered test favours Double: final Q -bias 0.06056 versus 0.07451 for Q -learning (difference -0.01395 , $p = 3.680 \times 10^{-5}$). But both values are positive, i.e. both agents still rate the left action higher than its true value -0.1 at the end of 300 episodes. A likely reason is the zero initialisation: the true value is below the initial value, and the left action is taken only about $\epsilon/2$ of the time late in training. The warm-start ablation (Sec. VI) removes this transient and gives a cleaner contrast.

b) H2 (trap MDP, actions): supported.: The suboptimal-action fraction is 0.1186 for Double versus 0.3795 for Q -learning ($p = 2.899 \times 10^{-10}$); in the first 20 episodes it is 0.4401 versus 0.8054. The peak start-state overestimate, which is the textbook signature, is 0.1060 for Q -learning and 0.008322 for Double ($p = 5.899 \times 10^{-9}$). Weighted Double behaves like Double (peak 0.008564). Maxmin keeps the bias small (0.01112) but takes more suboptimal actions than Double (0.1967): a possible cause is that two estimators with a minimum are slow to rank the actions at B, which we did not isolate.

c) H3 (random MDP): first part refuted, second part supported.: Q -learning’s final start-state bias is -0.1498 , i.e. negative, not positive, and Double Q -learning’s is

TABLE I

Main results (mean $\pm$ std over 5 seeds; each seed is the mean of 1,000 independent runs). `bias_first`: start-state estimated-minus-true, mean over the first 20 episodes; `bias_final` and `qbias_final`: last 20 episodes; `subopt_all`: suboptimal-action fraction over all episodes; `greedy_err_final`: greedy-policy error at the end. On the trap MDP bias is clipped at zero from below (Sec. III).

Method	Task	bias_first $\downarrow$	bias_final $\downarrow$	qbias_final $\downarrow$	subopt_all $\downarrow$	greedy_err_final $\downarrow$	n
Q-learning	maxbias	0.038 $\pm$ 0.001	0.008 $\pm$ 0.002	0.075 $\pm$ 0.002	0.380 $\pm$ 0.005	0.083 $\pm$ 0.013	5
Weighted Double Q-learning	maxbias	0.003 $\pm$ 0.000	0.001 $\pm$ 0.000	0.073 $\pm$ 0.000	0.127 $\pm$ 0.002	0.020 $\pm$ 0.004	5
Maxmin Q-learning	maxbias	0.000 $\pm$ 0.000	0.001 $\pm$ 0.000	0.080 $\pm$ 0.000	0.197 $\pm$ 0.002	0.024 $\pm$ 0.002	5
Double Q-learning	maxbias	0.003 $\pm$ 0.000	0.001 $\pm$ 0.000	0.061 $\pm$ 0.001	0.119 $\pm$ 0.002	0.017 $\pm$ 0.002	5
Q-learning	random20	-2.960 $\pm$ 0.022	-0.150 $\pm$ 0.011	-0.444 $\pm$ 0.005	0.255 $\pm$ 0.002	0.173 $\pm$ 0.016	5
Weighted Double Q-learning	random20	-3.283 $\pm$ 0.016	-0.360 $\pm$ 0.003	-0.839 $\pm$ 0.005	0.313 $\pm$ 0.003	0.257 $\pm$ 0.010	5
Maxmin Q-learning	random20	-3.428 $\pm$ 0.021	-0.721 $\pm$ 0.018	-1.175 $\pm$ 0.009	0.282 $\pm$ 0.002	0.228 $\pm$ 0.026	5
Double Q-learning	random20	-3.281 $\pm$ 0.017	-0.364 $\pm$ 0.007	-0.840 $\pm$ 0.005	0.312 $\pm$ 0.001	0.254 $\pm$ 0.011	5

lower still (-0.3636 ; difference -0.2138 , $p = 4.841 \times 10^{-9}$). The all-pair Q -bias shows the same ordering (Q-learning -0.4437 , Double -0.8404). Double does not take fewer suboptimal actions: 0.3119 versus 0.2551 for Q-learning ($p = 1.642 \times 10^{-10}$), and its final greedy error is higher (0.2538 versus 0.1729). Maxmin has the most negative bias (-0.7212) but a lower suboptimal fraction than Double (0.2822). Weighted Double is indistinguishable from Double here (bias -0.3597 , suboptimal fraction 0.3127). On this MDP family and at this budget, removing the positive bias therefore did not help; random20 is a regime where Q-learning is the best of the four on action quality, a failure case for double estimators.

d) What the numbers do not show.: The main table shows that the start-state estimates are below the true values for every method on random20, but it does not say why: at the end of training the zero initialisation, the per-table update rate and adaptive sampling all push the same way. The ablations below address each separately, and none of them is a proof of mechanism. The tests use $n = 5$ seed means with very small spread, so the p -values are small for almost any difference; effect sizes in rh compare are large (e.g. Cohen’s $d = 42.60$ for the suboptimal fraction on random20), and statistical significance here says only that the difference is reproducible across seeds of this MDP family.

VI. Ablations and Analysis

a) Warm start at the true values (Table II).: Initialising Q at Q^* removes the transient from zero initialisation. On the trap MDP the final Q -bias of Q-learning is 0.07478 (Q-learning has pulled $Q(A, \text{left})$ well above its true value of -0.1 even though it started at the truth), whereas Double’s is 0.006783, and the suboptimal fraction falls to 0.05195 versus 0.1008 ($p = 3.063 \times 10^{-7}$). This is the clean version of H1: Double nearly removes the overestimation, Q-learning does not. On random20 with warm start, the first-20-episode start-state bias is slightly positive for Q-learning (0.003180), but by the end both estimators are below the truth (Q-learning -0.05327 , Double -0.1004), so the negative final bias on random MDPs is not only a transient from the initial values. Notably the action

ranking flips relative to the zero-initialised main run: with warm start Double takes fewer suboptimal actions than Q-learning (0.1440 versus 0.1638, $p = 3.553 \times 10^{-8}$). The cost of Double in the main random20 run is thus a cost of reaching the true values from a bad initialisation, not a property of the steady state.

b) Simultaneous double update (Table III).: Updating both tables on every sample with the cross targets gives, on the trap MDP, exactly the Q-learning numbers (peak bias 0.1060 versus 0.1060; suboptimal fraction 0.3795 versus 0.3795), and on random20 nearly the same (final bias -0.1465 versus -0.1498). This follows from the symmetry argument of Section III: cross-evaluation only helps when the two tables are fitted on disjoint samples. The small random20 differences come from the in-place update order, which we did not verify beyond this reasoning.

c) Step size, noise and exploration on random20 (Table IV, Fig. 2).: The step-size sweep suggests that much of Double’s penalty on random20 is a per-table learning-speed effect: Q-learning with half the step size ($\alpha = 0.05$) has final bias -0.3425 and suboptimal fraction 0.3173, close to Double at $\alpha = 0.1$ (-0.3636 and 0.3119); at $\alpha = 0.2$ the gap in action quality shrinks (Q-learning 0.2348, Double 0.2524). This is consistent with, but does not prove, an effective-step-size explanation (each Double table sees half the updates). The noise sweep does not support the first part of H4: Q-learning’s final bias is not increasing in σ at the grid points (-0.1806 , -0.1498 , -0.2089 , -0.5266 for $\sigma = 0.25, 1, 2, 4$), and more noise makes both estimators more negative at $\sigma = 4$. The ranking of action quality does change with noise: Q-learning is better at $\sigma \leq 2$ (suboptimal fraction 0.2777 versus 0.2985 at $\sigma = 2$) but Double is better at $\sigma = 4$ (0.3349 versus 0.3767), the only swept regime of random20 where the double estimator takes fewer suboptimal actions.

d) Exploration and adaptive sampling.: With $\epsilon = 1$ (uniformly random behaviour, so the data do not depend on the estimates) Q-learning’s final bias is positive (0.09032) while Double’s is negative (-0.09331): this is the textbook sign pattern. With $\epsilon = 0.1$ both are negative, and the negativity shrinks as ϵ grows (Q-learning: -0.1498 ,

TABLE II
Warm-start ablation (initial $Q = Q^*$), mean $\pm$ std over 5 seeds. Same metrics and conventions as Table I.

Method	Task	bias_first $\downarrow$	bias_final $\downarrow$	qbias_final $\downarrow$	subopt_all $\downarrow$	n
Q-learning	random20	0.003 ± 0.003	-0.053 ± 0.005	-0.098 ± 0.001	0.164 ± 0.002	5
Double Q-learning	random20	0.001 ± 0.002	-0.100 ± 0.006	-0.107 ± 0.002	0.144 ± 0.002	5
Maxmin Q-learning	random20	-0.120 ± 0.002	-0.507 ± 0.002	-0.464 ± 0.001	0.164 ± 0.002	5
Weighted Double Q-learning	random20	0.002 ± 0.001	-0.069 ± 0.006	-0.078 ± 0.002	0.141 ± 0.001	5
Q-learning	maxbias	0.000 ± 0.000	0.011 ± 0.001	0.075 ± 0.001	0.101 ± 0.002	5
Double Q-learning	maxbias	0.000 ± 0.000	0.000 ± 0.000	0.007 ± 0.001	0.052 ± 0.000	5
Maxmin Q-learning	maxbias	0.000 ± 0.000	0.000 ± 0.000	0.004 ± 0.000	0.050 ± 0.000	5
Weighted Double Q-learning	maxbias	0.000 ± 0.000	0.000 ± 0.000	0.009 ± 0.001	0.052 ± 0.000	5

TABLE III
Simultaneous double update (both tables updated on each sample), mean $\pm$ std over 5 seeds.

Method	Task	bias_peak $\downarrow$	bias_final $\downarrow$	subopt_all $\downarrow$	n
Double Q-learning (both tables)	maxbias	0.106 ± 0.002	0.008 ± 0.002	0.380 ± 0.005	5
Double Q-learning (both tables)	random20	-0.133 ± 0.011	-0.147 ± 0.012	0.255 ± 0.002	5

-0.03745 , 0.09032 for $\epsilon = 0.1, 0.3, 1$). This pattern is consistent with the negative bias on random MDPs being caused by greedy, adaptively collected data (an action that looks bad early stops being sampled), but ϵ is the only factor we varied here, so we state it as a possibility rather than a finding.

e) Number of actions at the trap state (Table V): Q-learning’s peak overestimate grows with the number of actions, from 0.03665 at $n = 2$ to 0.1281 at $n = 100$ (all five grid values are in the table and the sequence increases), while Double stays between 0.006169 and 0.01011 ; the second part of H4 is supported on the trap MDP at these grid points. In action quality the trap grows costly for Q-learning (suboptimal fraction 0.9404 at $n = 100$) and for Maxmin ($N = 2$: 0.7938 , versus 0.1280 for Double). Maxmin’s bias is small but its action quality degrades with n , a failure case of the two-estimator minimum that we did not investigate further.

VII. Limitations

This is a small, CPU-only study and its scope is narrow. (i) One MDP family: random20 has 20 states, 4 actions and 3 successors per pair, with a fixed discount of 0.7 and reward means from a standard normal; the conclusions about its bias sign and about which method takes fewer suboptimal actions may change with different gap-to-noise ratios, branching or discount, and our σ sweep already shows the action-quality ranking changing within this family. (ii) The random-MDP configuration (discount, episode length, random start states, 1,000 episodes) was fixed after a pilot on seed 0; a discount of 0.9 with the same budget was not converged, so our random20 results describe a discount of 0.7 and a finite budget, not the asymptotic behaviour. (iii) Hyper-parameters are not tuned for any method ($\alpha = 0.1$, $\epsilon = 0.1$ throughout); a fair per-method tuning could reorder Q-learning, Double

and Maxmin on random20, and our step-size sweep covers only Q-learning and Double. Weighted Double ($c = 1$) and Maxmin ($N = 2$) use one fixed setting each. (iv) All baselines are our own reimplementations from the published descriptions; we did not compare against reference code. (v) Only 5 seeds are used for tests, each the mean over 1,000 runs; this makes the between-seed spread tiny and the p -values small, but it measures reproducibility over seeds of one generator, not generalisation to other MDP families. (vi) The mechanisms offered in Section VI (effective step size, adaptive sampling) are supported only by one-factor sweeps; we did not run an experiment that fixes the data (e.g. off-policy learning from a common fixed dataset) or that equalises the number of updates per table. (vii) The trap-MDP start-state bias is clipped at zero by construction, so it cannot register underestimation, and the signed Q -bias is dominated by the zero initialisation unless warm-started. (viii) The simultaneous-update ablation is an in-place sequential implementation whose small random20 deviation from Q-learning is attributed to update order without a dedicated test. (ix) Tabular results do not carry over to function approximation, where bias can compound through bootstrapping. A fuller study would add more MDP families, a discount and budget grid with convergence checks, per-method tuning, many more seeds, and function approximation.

VIII. Conclusion

On the textbook maximisation-bias MDP our reimplementations reproduces the known behaviour: Q-learning overestimates the start-state value and takes the trap action far more often than Double Q-learning, and the effect grows with the number of actions at the trap state. On random 20-state MDPs the same comparison does not carry over at our budget. All four estimators underestimate the start-state value at the end of training, Q-learning

TABLE IV

Sensitivity on random20 (means over 5 seeds). Q = Q-learning, DQ = Double Q-learning; $\hat{V}$ -bias: final start-state estimated-minus-true; $\hat{Q}$ -bias: final all-pair $\hat{Q}$ -bias; subopt: suboptimal fraction over all episodes. The default setting of each sweep ($\sigma = 1$, $\epsilon = 0.1$, $\alpha = 0.1$) is the main-run value.

Sweep	value	$\hat{V}$ -bias Q	$\hat{V}$ -bias DQ	$\hat{Q}$ -bias Q	$\hat{Q}$ -bias DQ	subopt Q	subopt DQ
sigma	0.25	-0.181	-0.433	-0.452	-0.876	0.276	0.348
	1.0	-0.150	-0.364	-0.444	-0.840	0.255	0.312
	2.0	-0.209	-0.452	-0.563	-0.924	0.278	0.298
	4.0	-0.527	-1.156	-1.211	-1.658	0.377	0.335
epsilon	0.1	-0.150	-0.364	-0.444	-0.840	0.255	0.312
	0.3	-0.037	-0.126	-0.087	-0.255	0.329	0.344
	1.0	0.090	-0.093	0.068	-0.114	0.750	0.750
alpha	0.02	-0.699	-0.966	-1.377	-1.680	0.378	0.388
	0.05	-0.342	-0.622	-0.829	-1.255	0.317	0.368
	0.1	-0.150	-0.364	-0.444	-0.840	0.255	0.312
	0.2	-0.123	-0.288	-0.307	-0.560	0.235	0.252

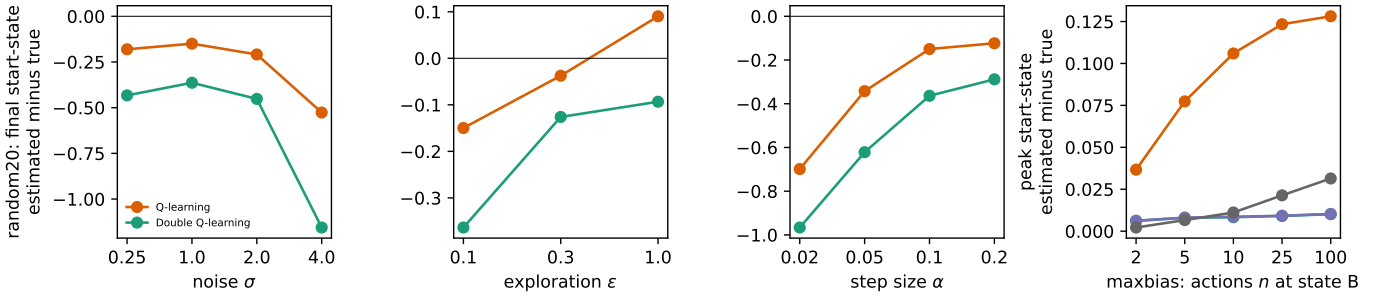


Fig. 2. Sensitivity sweeps (means over 5 seeds). First three panels: random20 final start-state bias of Q-learning (orange) and Double (green) versus σ , ϵ , α . Right: trap MDP peak start-state overestimate versus number of actions n at state B for all four systems (orange Q, green Double, purple Weighted Double, grey Maxmin).

TABLE V

Trap MDP with n actions at state B (means over 5 seeds). Q, DQ, WDQ, MM: Q-learning, Double, Weighted Double, Maxmin. Left block: peak start-state overestimate; right block: suboptimal fraction over all episodes.

n	start-state peak bias				suboptimal fraction			
	Q	DQ	WDQ	MM	Q	DQ	WDQ	MM
2	0.037	0.006	0.006	0.002	0.142	0.103	0.106	0.082
5	0.077	0.008	0.008	0.007	0.246	0.111	0.117	0.126
10	0.106	0.008	0.009	0.011	0.380	0.119	0.127	0.197
25	0.123	0.009	0.009	0.021	0.675	0.122	0.133	0.380
100	0.128	0.010	0.010	0.031	0.940	0.128	0.137	0.794

the least, and Double Q-learning takes more suboptimal actions than Q-learning. Ablations are consistent with this being a finite-time effect of slower per-table learning and of greedy, adaptively collected data: with a warm start Double takes fewer suboptimal actions, with uniformly random behaviour the textbook sign pattern (Q-learning positive, Double negative) appears, and updating both tables on the same sample removes the benefit of Double entirely. Maxmin Q-learning with two estimators controls the bias at the price of action quality in the larger-action-space trap. Our evidence is limited to small tabular MDPs, one noise model and untuned shared hyper-parameters, so

the practical message is cautious: judge a bias correction by the policy it yields and by the training regime, not by the sign of the bias alone.

References

- [1] J. E. Smith and R. L. Winkler, “The Optimizer’s Curse: Skepticism and Postdecision Surprise in Decision Analysis,” *Management Science*, 2006. [Online]. Available: <https://doi.org/10.1287/mnsc.1050.0451>
- [2] Z. Zhang, Z. Pan, and M. J. Kochenderfer, “Weighted Double Q-learning,” *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 2017. [Online]. Available: <https://doi.org/10.24963/ijcai.2017/483>
- [3] Z. Ren, G. Zhu, H. Hu, B. Han, J. Chen, and C. Zhang, “On the Estimation Bias in Double Q-Learning,” *arXiv preprint arXiv:2109.14419*, 2021. [Online]. Available: <https://arxiv.org/abs/2109.14419>
- [4] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” *arXiv preprint arXiv:1509.06461*, 2015. [Online]. Available: <https://arxiv.org/abs/1509.06461>
- [5] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, “Rainbow: Combining Improvements in Deep Reinforcement Learning,” *arXiv preprint arXiv:1710.02298*, 2017. [Online]. Available: <https://arxiv.org/abs/1710.02298>
- [6] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” *arXiv preprint arXiv:1802.09477*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.09477>

- [7] Q. Lan, Y. Pan, A. Fyshe, and M. White, “Maxmin Q-learning: Controlling the Estimation Bias of Q-learning,” arXiv preprint arXiv:2002.06487, 2020. [Online]. Available: <https://arxiv.org/abs/2002.06487>
- [8] A. Cini, C. D’Eramo, J. Peters, and C. Alippi, “Deep Reinforcement Learning with Weighted Q-Learning,” arXiv preprint arXiv:2003.09280, 2020. [Online]. Available: <https://arxiv.org/abs/2003.09280>
- [9] M. Sabry and A. M. A. Khalifa, “On the Reduction of Variance and Overestimation of Deep Q-Learning,” arXiv preprint arXiv:1910.05983, 2019. [Online]. Available: <https://arxiv.org/abs/1910.05983>
- [10] A. Kuznetsov, A. Grishin, A. Tsypin, A. Ashukha, A. Kadurin, and D. Vetrov, “Automating Control of Overestimation Bias for Reinforcement Learning,” arXiv preprint arXiv:2110.13523, 2021. [Online]. Available: <https://arxiv.org/abs/2110.13523>
- [11] J. Wagenbach and M. Sabatelli, “Factors of Influence of the Overestimation Bias of Q-Learning,” arXiv preprint arXiv:2210.05262, 2022. [Online]. Available: <https://arxiv.org/abs/2210.05262>